

Code-Layout, Readability and Reusability

Date	15 March 2024
Team ID	
Project Name	OptiCrop
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout	Yes	PEP 8 4-space indents
2	Proper File Structure	Files and folders are logically organized	Yes	app.py / model_pipeline.py split
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. crop_input, prediction
4	Function / Method Names	Functions are descriptively named	Yes	predict(), train_model()
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings & inline notes
6	Modular Design	Code is split into reusable functions/modules	Yes	Reusable helper functions
7	No Redundant Code	Duplicate or unused code is removed	Yes	No dead/duplicate code
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/except on prediction

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	load_model()	Python / pickle	app.py	High

2	<code>predict_crop()</code>	Python / sklearn	<code>app.py</code> routes	High
3	<code>preprocess()</code>	Pandas / NumPy	<code>model_pipeline.py</code>	Medium
4	<code>plot_eda()</code>	Matplotlib/Seaborn	<code>model_pipeline.py</code>	Medium
5				

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clean modular layout
Readability	5	Descriptive naming, comments
Reusability	4	Helper functions reused
Documentation / Comments	4	Docstrings present
Overall Score	5	High overall quality